PROJECT PRESENTATION

CloudPulse

Cloud Infrastructure Operations

A unified view of resources, estimated spend, and incidents

Srujan Gowda KS • 24BCS10339

BSc Computer Science | Scaler School of Technology

GitHub: https://github.com/srujangowda07/devops-project



The problem and the idea

Cloud resources and operational issues can be hard to follow in one place.

THE CHALLENGE

Scattered information

Resource status, provider, region, and estimated cost need a clear overview.

Incidents are easy to miss

Operational issues need a visible place to be recorded and reviewed.

THE CLOUDPULSE APPROACH

Unified inventory

One dashboard lists registered resources and their status.

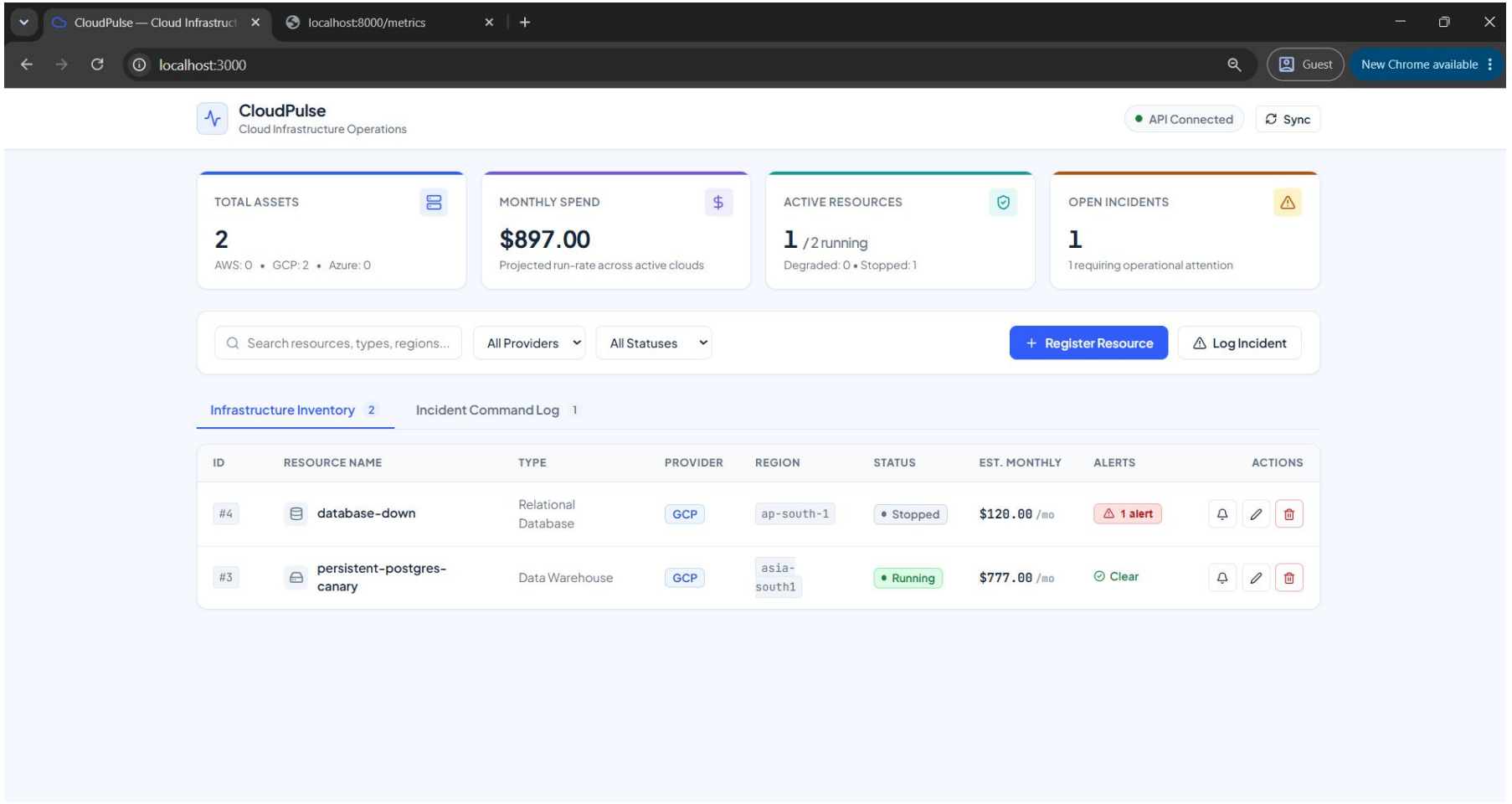
Incident visibility

Open incidents and estimated monthly spend appear alongside inventory.

Important: costs shown in the app are estimates entered for tracked resources, not live cloud billing data.

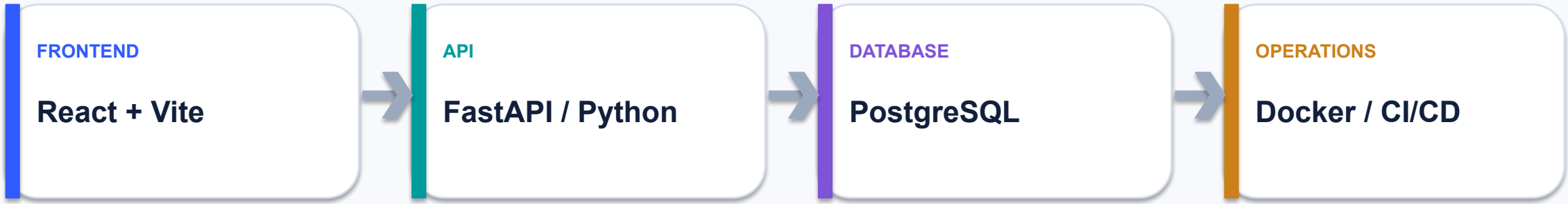
The CloudPulse dashboard

The interface brings inventory, estimated spend, resource status, and incidents together.



A simple, layered architecture

Each part has a clear job, so the app can be developed and run as separate services.



REQUEST FLOW

The browser calls the API; the API reads or updates records in PostgreSQL. Docker Compose runs the services together for local development.

Deployment and monitoring tools support delivery and operations; they do not make the inventory a live cloud-provider feed.

Tools chosen for the job

A practical stack that covers the application and its delivery workflow.

React + Vite

Builds a responsive browser interface.

FastAPI + Python

Provides API endpoints and interactive API docs.

PostgreSQL + SQLAlchemy

Stores application records in a relational database.

Docker Compose

Starts the frontend, backend, database, and migration service together.

GitHub Actions + Trivy + GHCR

Automates checks, scans images, and publishes container images.

Terraform + Kubernetes + Helm

Defines infrastructure and packages app deployment configuration.

Prometheus + Grafana

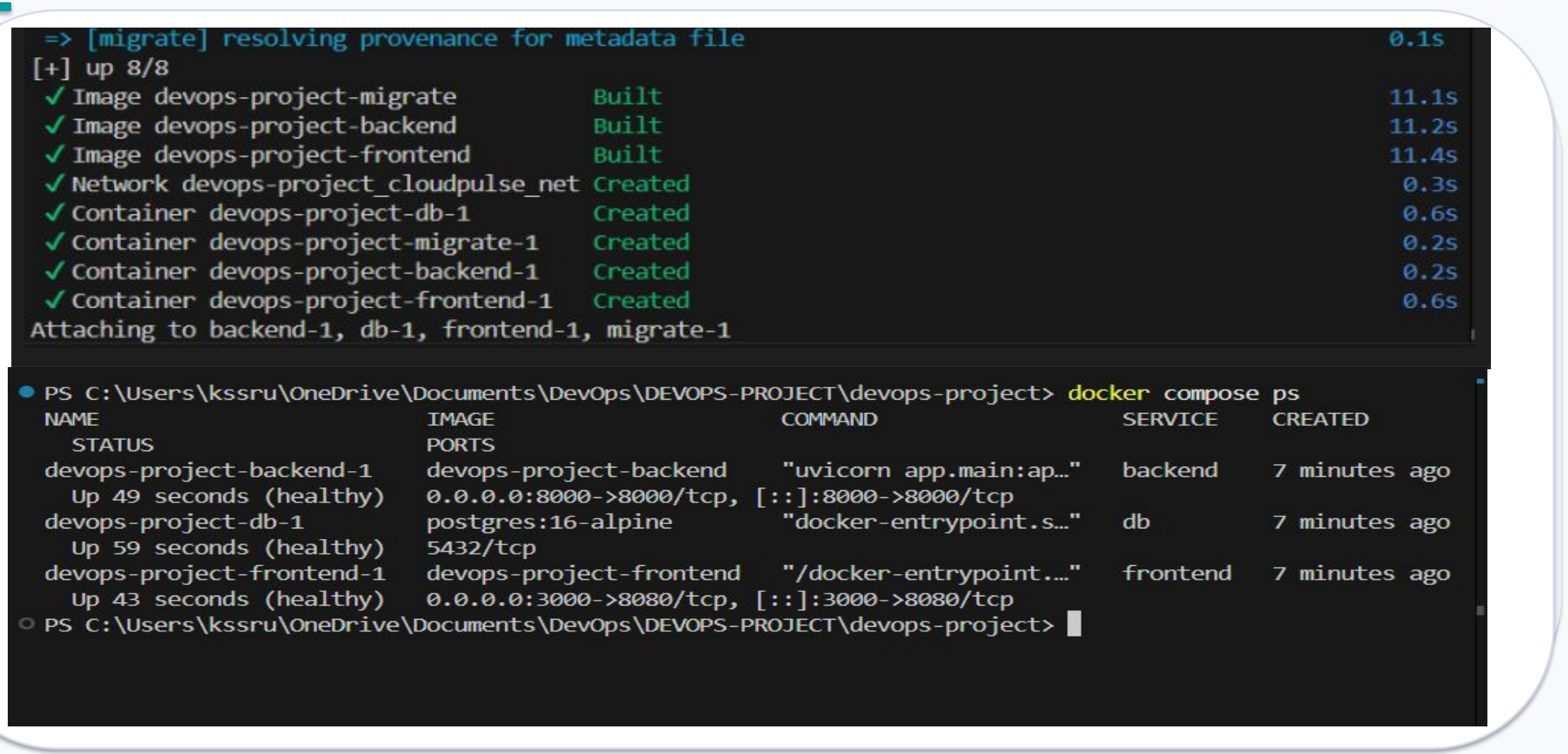
Provides metrics collection and dashboard visualization configuration.

Why this stack? It keeps the web app, data layer, testing, packaging, security checks, and deployment configuration clearly separated.

Alternatives exist (for example, Django or Flask for the API), but this project uses FastAPI for a lightweight API and built-in interactive documentation.

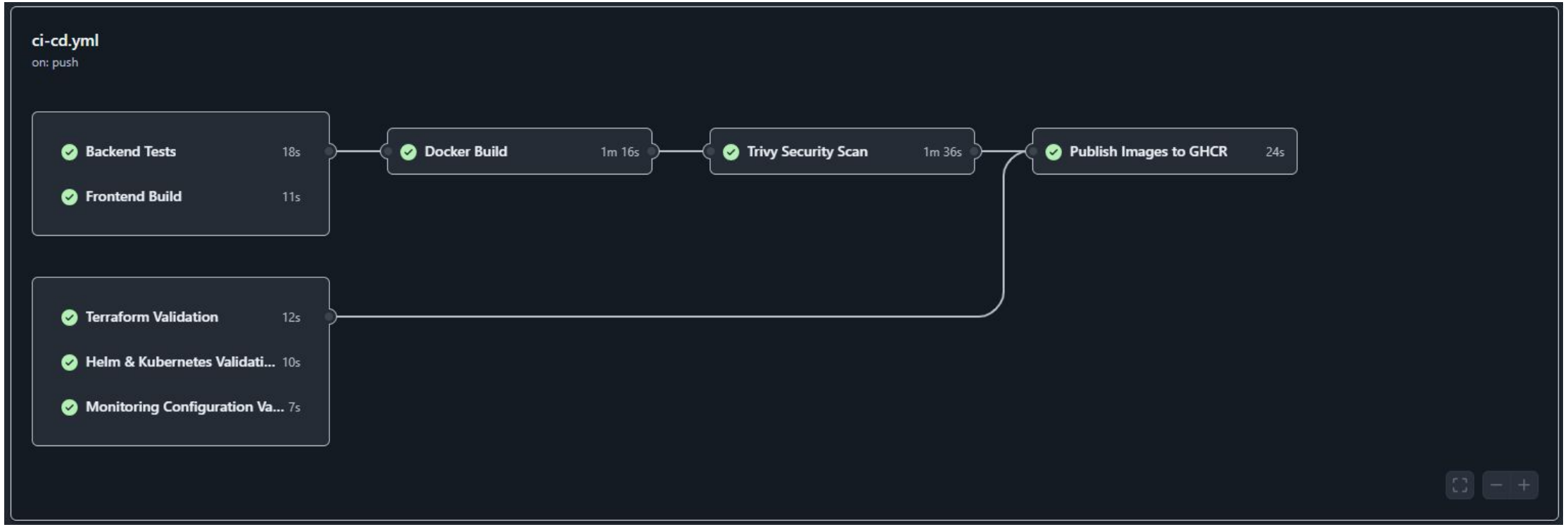
Running the app with Docker Compose

The screenshot shows images built and the application services created for the local environment.



Automated CI/CD checks

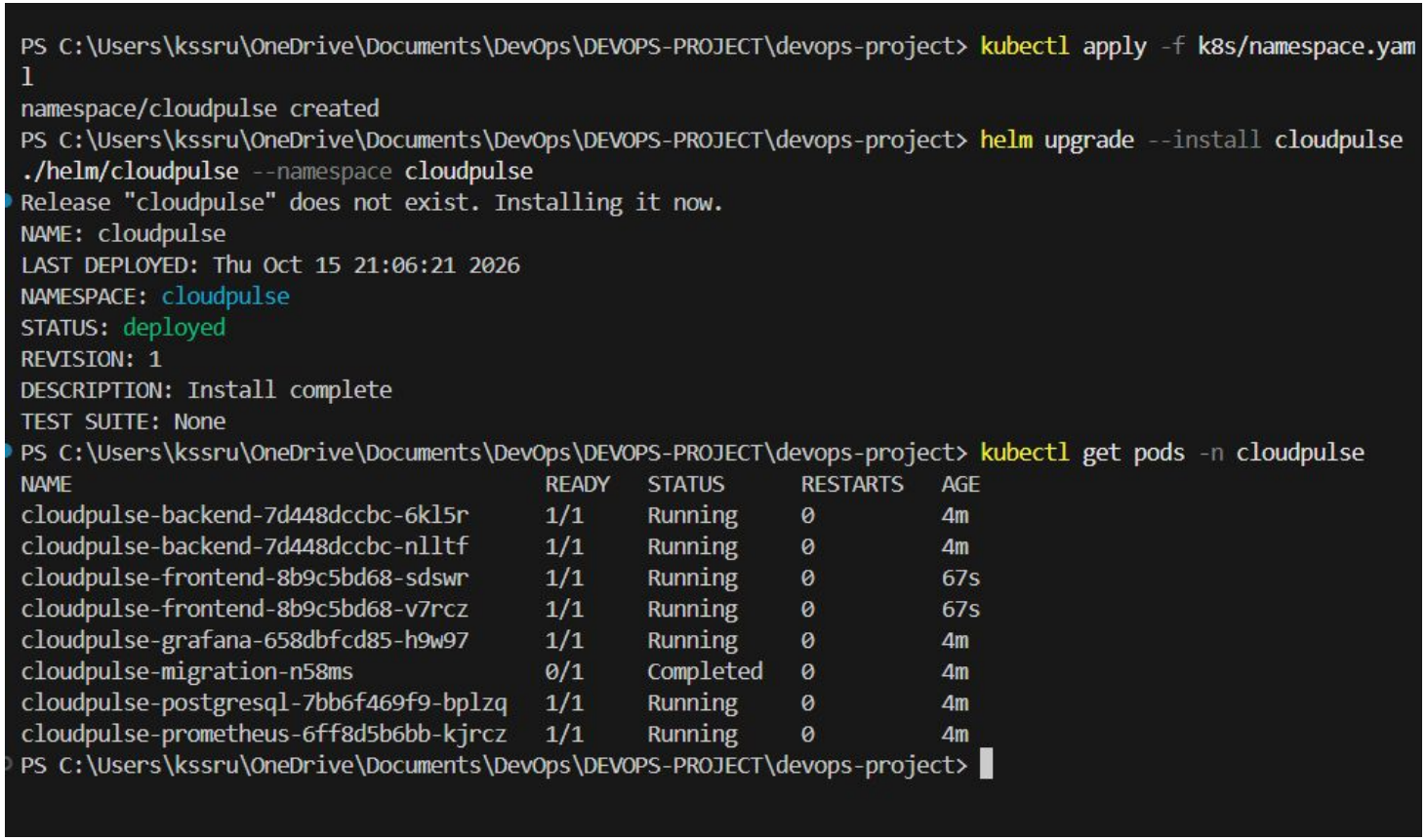
The GitHub Actions workflow shows tests, builds, security scanning, validation, and image publishing.



Evidence shown: all visible workflow jobs completed successfully, including Trivy scanning and publishing images to GHCR.

Kubernetes deployment

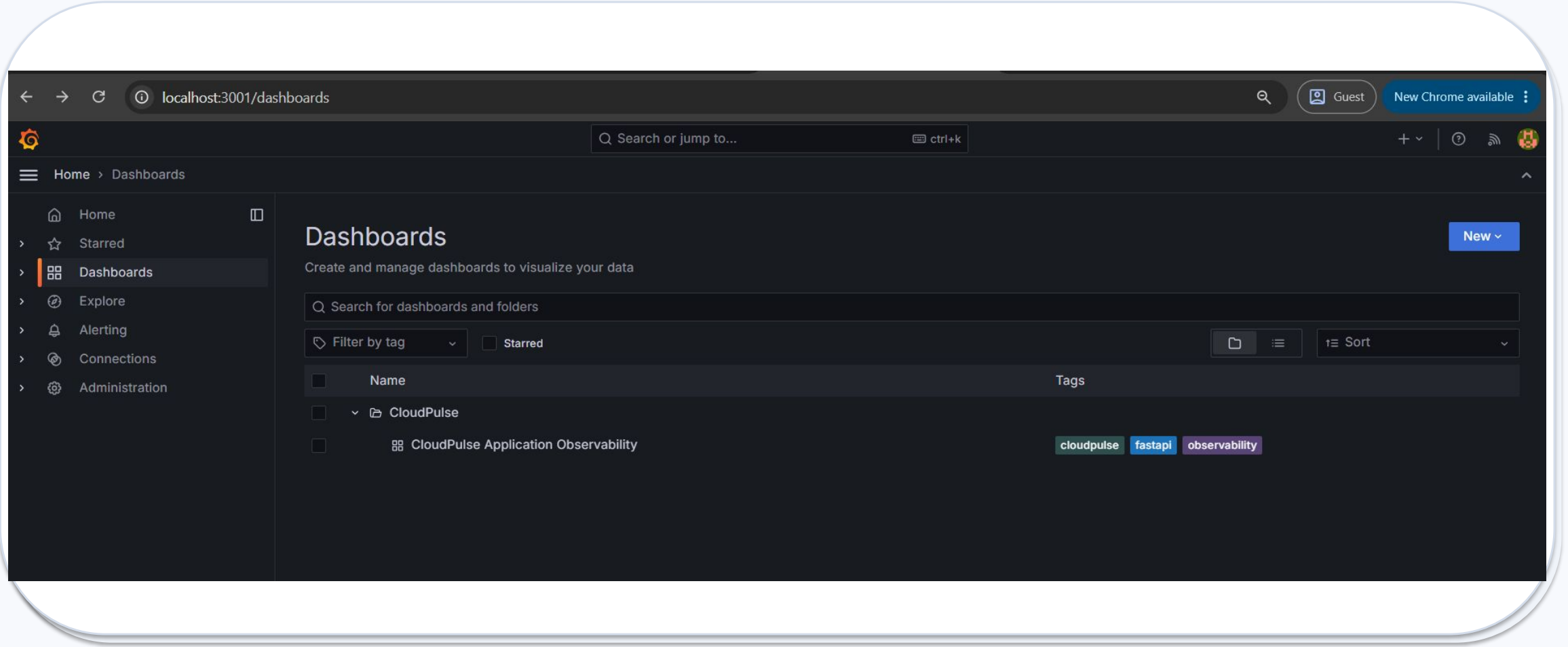
Helm reports a deployed release, and the screenshot shows the status of application and monitoring pods.



Evidence shown: CloudPulse release deployed in the cloudpulse namespace. The migration pod is shown as 0/1 at capture time; not every pod is ready.

Monitoring dashboard configured

Grafana contains a CloudPulse observability dashboard entry for application metrics.



Automated tests

The provided test output shows API lifecycle and metrics endpoint checks passing.

```
tests/test_api.py::test_delete_resource PASSED                                [ 78%]

tests/test_api.py::test_incident_lifecycle_and_relationship PASSED            [ 85%]

tests/test_api.py::test_incident_nonexistent_resource_fails PASSED            [ 92%]

tests/test_api.py::test_delete_resource PASSED                                [ 78%]
tests/test_api.py::test_incident_lifecycle_and_relationship PASSED            [ 85%]
tests/test_api.py::test_incident_nonexistent_resource_fails PASSED            [ 92%]
tests/test_api.py::test_metrics_endpoint PASSED                               [100%]

============================================ warnings summary ============================================
.venv\Lib\site-packages\fastapi\testclient.py:1
  C:\Users\kssru\OneDrive\Documents\DevOps\DEVOPS-PROJECT\devops-project\backend\.venv\Lib\site-package
\fastapi\testclient.py:1: StarletteDeprecationWarning: Using `httpx` with `starlette.testclient` is dep
ecated; install `httpx2` instead.
    from starlette.testclient import TestClient as TestClient  # noqa

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
================================== 14 passed, 1 warning in 0.29s ===================================
(.venv) PS C:\Users\kssru\OneDrive\Documents\DevOps\DEVOPS-PROJECT\devops-project\backend>
```